THE BEGINNER'S GUIDE

Building a Multilingual Website the Right Way

Internationalization (i18n) + International SEO, explained from zero.

What you will learn:

- The one big idea that makes i18n simple
- The 3 things you actually translate
- How to choose your URL strategy (/es vs subdomain)
- The golden rule: one config drives everything
- International SEO: hreflang, canonical, sitemap, lang
- How adding a new language becomes a 2-step job

Prepared for MyHomeStyler - 2026-06-04

1. Start Here: The One Big Idea

If you remember only one sentence from this guide, make it this:

> **THE BIG IDEA**
> Separate your CONTENT from your CODE. Your code should never contain sentences. Every piece of text lives in a separate translation file, looked up by a key.

A beginner writes this (text baked into the code):

```
<button>Sign Up</button>
```

An i18n-ready app writes this (text looked up by a key):

```
<button>{t("signUp")}</button>

// messages/en.json -> { "signUp": "Sign Up" }
// messages/es.json -> { "signUp": "Registrarse" }
```

Now the SAME code renders in any language. To add a language you add a translation file - you never touch the components. That separation is the whole game.

2. The 3 Things You Actually Translate

People think 'translation' is one job. It is really three, and beginners forget the last two.

1) UI strings
Buttons, menus, labels, form placeholders. Short, reused everywhere. These go in per-language JSON files (en.json, es.json).

2) Content and SEO copy
Page titles, meta descriptions, FAQs, blog posts, marketing text. This is what Google reads and ranks. It must be translated by a human, not just machine-translated.

3) The URLs themselves
This is the one beginners miss. A Spanish page should live at its own address, e.g. /es/pricing, not the English URL. Search engines treat each language URL as its own page to index.

> **WHY URLs MATTER**
> If English and Spanish share one URL, Google can only index one language. Real per-language URLs (/es/...) are what let you rank in Spanish search results.

3. Pick Your URL Strategy (decide this first)

There are three ways to give each language its own URL. Pick ONE and stay consistent.

Option A - Subdirectory: example.com/es/pricing (recommended)

- Easiest to set up and maintain - one website, one domain.
- All your SEO authority stays on one domain.

- Default choice for most sites, including this one.

Option B - Subdomain: es.example.com/pricing

- More separation, but Google may treat it as a partly separate site.
- More DNS and hosting setup.

Option C - Country domain: example.es/pricing

- Strongest local signal, best if you target a specific country hard.
- Most expensive and complex - you buy and manage many domains.

> **RECOMMENDATION**
> Start with Option A (subdirectory /es). It gives you 90% of the SEO benefit for 10% of the effort. Only move to B or C when you have a strong country-specific reason.

4. The Golden Rule: One Source of Truth

This is the rule that decides whether adding language #4 takes 2 minutes or 2 days.

> **THE GOLDEN RULE**
> Define your list of languages in ONE place. Every other part of the system - routing, the language switcher, the sitemap, SEO tags - should READ from that one list, never hardcode a language.

Bad (a beginner trap): the language is checked by hand in dozens of files.

```
// scattered in 15 different files...
if (locale === "ar") { ... } else { ... }
const url = locale === "ar" ? "/ar" + path : path;
```

Why it is bad: to add Spanish you must hunt down and edit every one of those lines. Miss one and that page silently breaks.

Good: one config, and helpers that loop over it.

```
// config.ts - the single source of truth
export const LOCALES = ["en", "es", "ar"];
export const DEFAULT = "en";

// everything else is generated from LOCALES
LOCALES.map(l => buildUrl(path, l));   // sitemap, hreflang, switcher...
```

Now adding a language is adding ONE item to that array. Nothing else changes.

5. A Clean Folder Structure (Next.js example)

This project uses Next.js with the next-intl library. A tidy i18n setup looks like this:

```
/i18n
   routing.ts      <- the LOCALES list (single source of truth)
   request.ts      <- loads the right messages file
/messages
   en.json         <- English UI strings
   es.json         <- Spanish UI strings (same keys!)
   ar.json
/app/[locale]/    <- pages that get a real /es/... URL
   page.tsx        (home)
   pricing/...
middleware.ts     <- routes visitors to the right language
sitemap.ts        <- generated from LOCALES
robots.ts
```

The key idea: the [locale] folder name is a variable. One set of page files serves every language, and the URL prefix (/es) decides which translation loads.

6. Storing Translations the Right Way

Keep one JSON file per language. Every file has the SAME keys, just different values. Group keys into namespaces so they stay organized.

```
// en.json                    // es.json
{                             {
  "nav": {                      "nav": {
    "pricing": "Pricing",         "pricing": "Precios",
    "signUp": "Sign Up"           "signUp": "Registrarse"
  }                             }
}                             }
```

- Same keys in every file = a missing translation is easy to spot.
- One file per language = a translator can fill it in without touching code.
- Never put a full sentence in your components - only keys.

7. International SEO: The Checklist

Translating the page is half the job. The other half is telling search engines about it. Miss these and your translated pages will not rank.

a) The html lang attribute

Every page must declare its language so browsers and Google know.

```
<html lang="es">           (and dir="rtl" for Arabic/Hebrew)
```

b) hreflang tags - the most important one

On every page, list ALL its language versions, plus x-default (the fallback). This tells Google these pages are translations of each other, not duplicates.

```
<link rel="alternate" hreflang="en"    href="https://site.com/pricing" />
<link rel="alternate" hreflang="es"    href="https://site.com/es/pricing" />
<link rel="alternate" hreflang="x-default" href="https://site.com/pricing" />
```

c) One canonical per page
Each language URL points its canonical to itself - so /es/pricing is the canonical for the Spanish page, not the English one.

d) A localized sitemap
Your sitemap should list every language URL and cross-link them with the same hreflang alternates. This is how Google discovers your /es pages.

e) robots rules
Allow crawlers to reach your language paths (/es), and keep private areas (dashboard, admin) blocked under EVERY language prefix.

f) Translate the meta, not just the body
The page <title> and meta description must be translated too - that is the text shown in Google results. A Spanish page with an English title looks broken to searchers.

> **DO NOT MACHINE-TRANSLATE SEO COPY**
> Use human translation for titles, descriptions and important content. Google can detect low-quality auto-translation, and it reads unnaturally to real users - both hurt ranking and trust.

8. The Dream: Adding Language #4

If your architecture is right, adding French should be this short:

```
Step 1:  add "fr" to the LOCALES array.
Step 2:  add messages/fr.json with the French text.

That is it. Routing, the switcher, the sitemap,
hreflang and SEO tags all update automatically.
```

> **THE LITMUS TEST**
> If adding a language forces you to edit many files and hunt for 'if locale == ...' checks, your locale logic is not centralized yet. That is the #1 thing to refactor before scaling to more languages.

9. Common Beginner Mistakes

- Hardcoding text in components instead of using translation keys.
- Scattering 'if (locale === x)' across many files instead of one config.
- Forgetting hreflang and canonical tags (pages get treated as duplicates).
- Machine-translating titles and descriptions (hurts ranking and trust).
- Translating the visible page but leaving the meta title/description in English.
- Forgetting to set the html lang (and dir for right-to-left languages).
- Not blocking private pages (dashboard, admin) under the new language prefix.
- Mixing URL strategies (some pages /es, some on a subdomain) with no plan.

10. Right-to-Left Languages (Arabic, Hebrew)

These read right-to-left, so the whole layout must mirror. Two things handle most of it:

- Set dir="rtl" on the html element for those languages.
- Use logical CSS (margin-inline-start, not margin-left) so spacing flips automatically.

Latin languages like Spanish, French and German are left-to-right, so they need no special layout work - only translation.

11. Recommended Stack and Quick Glossary

For a Next.js site

- next-intl or next-i18next for messages and routing.
- App Router with an [locale] folder for real per-language URLs.
- A single LOCALES config that drives routing, sitemap and hreflang.

Glossary

- i18n - internationalization: building the app so it CAN support many languages.
- l10n - localization: actually translating and adapting for one language/region.
- locale - a language + region code, e.g. en-US, es-ES, ar-AE.
- hreflang - the tag that links a page to its other-language versions.
- canonical - the tag naming the one official URL for a page.
- slug - the readable part of a URL, e.g. /es/blog/mi-articulo.

12. One-Page Cheat Sheet

Setup order

- 1. Pick a URL strategy (use /es subdirectory).
- 2. Create ONE locales config - your single source of truth.
- 3. Move all text into per-language JSON files (same keys).
- 4. Build routing/sitemap/switcher that LOOP over the config.
- 5. Add SEO: html lang, hreflang + x-default, canonical, sitemap, robots.
- 6. Use human translation for titles, descriptions and content.

Add a new language (the goal)

- Add the code to the LOCALES array.
- Add one JSON translation file.
- Done - everything else is generated.

Build it config-driven from day one, and your tenth language is as easy as your second.